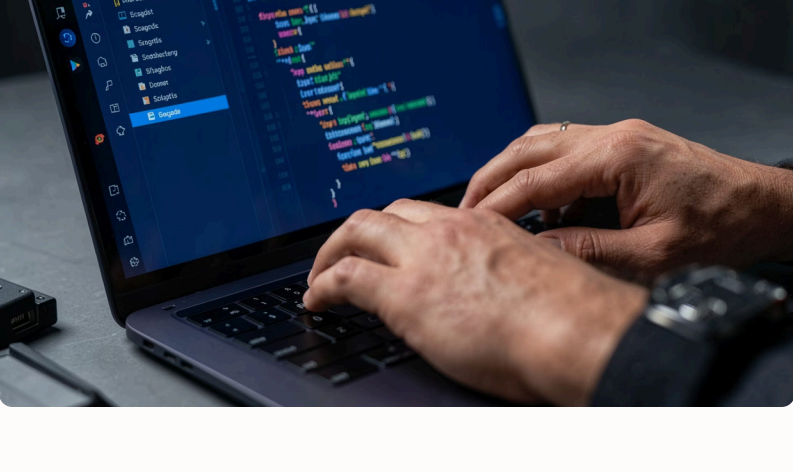


Hotel Room Reservations Application

Using Core Java & Object-Oriented Programming

Presented by Arjun Kumar Mahato — Roll No. RU-25-10269
B.Tech, Computer Science Engineering (AI & ML)
Rungta International Skills University — Session 2025–26

Project Snapshot



Type

Console-based application focused on core Java and OOP principles.

Technologies

Core Java, OOP, ArrayList, Scanner class — no external frameworks (prototype stage).

Purpose

Provide a reliable reservation workflow: view, book, manage and cancel room reservations while demonstrating OOP practices.

System Features & Workflow



View Available Rooms

List rooms with type, number, price and current status.



Book Room

Reserve by room number with customer details stored in-memory (ArrayList).



View Bookings

Display active bookings and associated customer information.



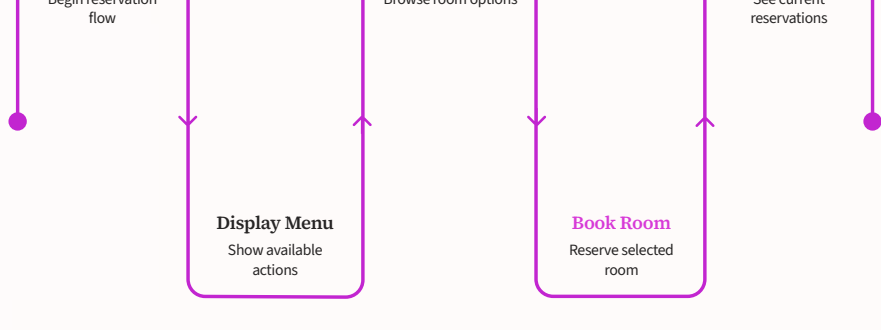
Cancel Booking

Release room, remove booking entry and update room status.



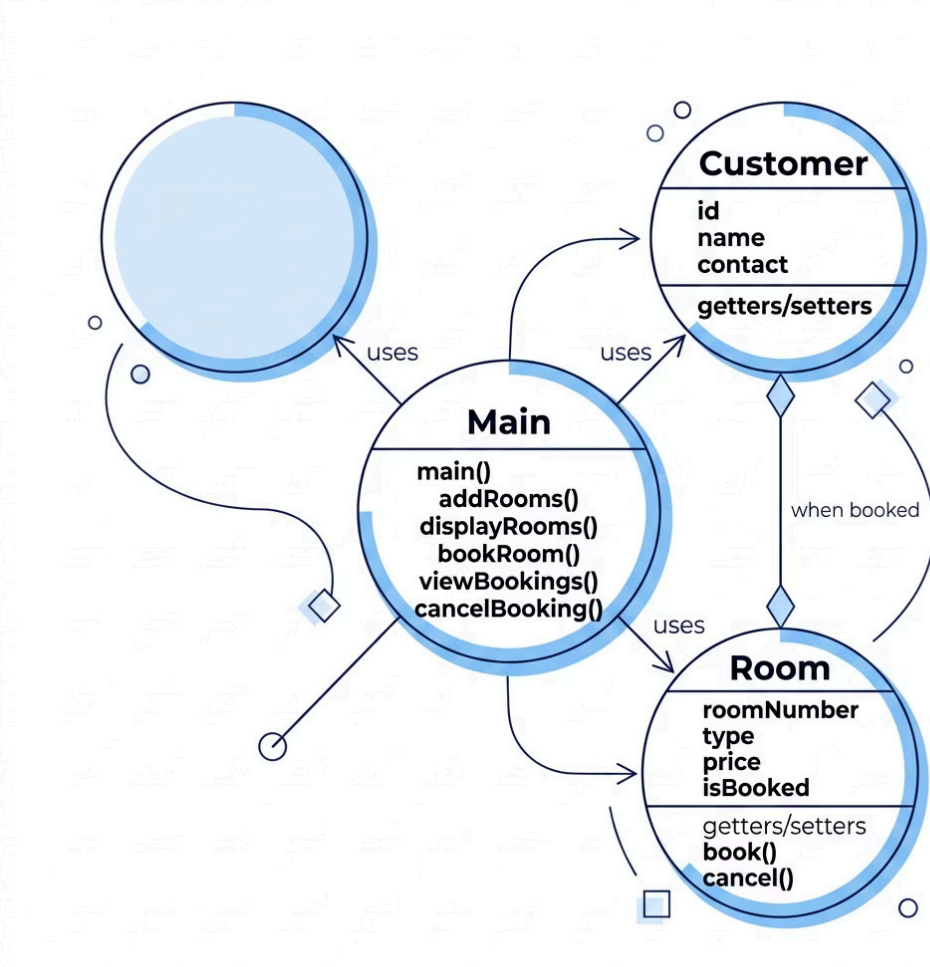
Customer Management

Create and reference customer records for each booking.



The console menu drives user interaction. Each selection calls a dedicated method (displayRooms(), bookRoom(), viewBookings(), cancelBooking()) that manipulates Room and Customer objects held in ArrayLists.

System Architecture & Core OOP Concepts



1

Class & Object

Classes (Customer, Room, Main) define structure; objects are runtime instances storing state.

2

Constructor

Initialises object fields — used in Room and Customer to set default state on creation.

3

Encapsulation

Private fields with public getters/setters protect internal state and control access.

4

Collections & I/O

ArrayList stores dynamic collections of rooms/bookings; Scanner reads user input from console.

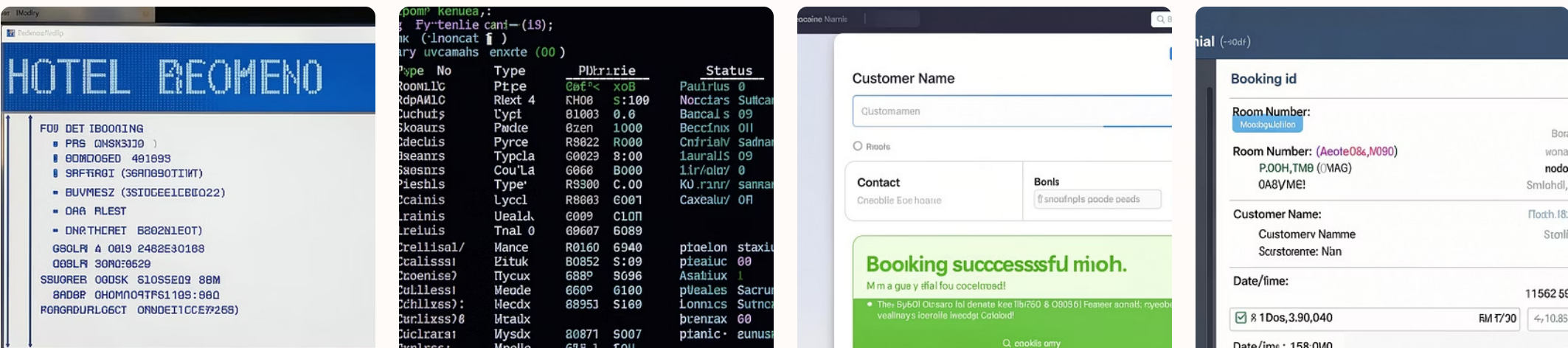
Source Code — Key Methods & Logic (Concise)

Below are concise summaries of primary methods implemented in Main with annotated purpose and control flow. (Slide-ready code snippets and callouts are prepared for the final .pptx export.)

01 addRooms() Creates Room objects with roomNumber, type, price; adds them to rooms ArrayList. Called once during initialisation.	02 displayRooms() Iterates rooms ArrayList and prints formatted table: room number, type, price, status (Available/Booked).	03 bookRoom() Validates roomNumber, checks isBooked flag, captures customer details via Scanner, marks room as booked, and appends booking to bookings ArrayList.
04 viewBookings() Displays active bookings with customer and room details, enabling quick verification.	05 cancelBooking() Validates booking existence, resets room.isBooked to false, removes booking record and confirms cancellation to user.	

Console Output Mockups

Realistic terminal screenshots are included in the deliverable to demonstrate expected user experience and flow.



Evaluation — Console Edge Cases & Advantages

1 Common Console Cases - Booking Cancellation — validates booking id before modifying state - Invalid Room Number — input validation with user-friendly error messages - Room Already Booked — prevents double-booking via isBooked flag - Graceful Exit — ensures resources (Scanner) are closed before termination	2 Advantages - User-friendly, minimal learning curve for terminal interaction - Fast processing and low resource footprint - Clear demonstration of OOP principles and modular design - Fully editable prototype ready for GUI/DB upgrade	Invalid room number, please try again. Error handling is implemented at input boundaries; further robustness can be added with try/catch blocks and input sanitisation.
---	--	---

Future Scope & Roadmap

1 MySQL Database Persistent storage for rooms, customers and booking history — supports concurrency and queries.	
2 Java Swing / JavaFX GUI Graphical front-end with responsive forms and visual room calendar.	
3 Online Payments & Auth Integrate payment gateway, add login/auth, role-based admin dashboard.	
4 Notifications & Reporting Email confirmations, PDF invoices, and booking analytics dashboard.	

Conclusion & Next Steps

This project is a compact, well-structured demonstration of Java OOP principles implemented as a functional hotel reservation prototype. The codebase is modular and ready for extension to persistent storage, GUI, and online features. For the viva: expect slides with annotated code, UML diagrams, terminal walkthroughs, and a migration plan to production-ready architecture.

Thank you

Presented by Arjun Kumar Mahato
RU-25-10269
Computer Science Engineering (AI & ML)
Rungta International Skills University

